KLAi

# Can I trust this model? Overfitting, cross-validation and leakage

*How do we know a model that predicts crusher energy demand will still work on a crusher it has never seen?*

CML3113 | Mining and Petroleum Engineering | Week 10, class 3 of 4 | Friday 20 November 2026, 08:00 to 11:00
KLAi | International University of East Africa | Jotham Martin Wambi

# Today in one view: what you will leave with

## By the end of today you can



Explain underfitting, a good fit and overfitting (the bias-variance trade-off), and tune the hyperparameter max_depth with a validation curve



Work out training error, validation error, the generalisation gap, and a cross-validation mean and standard deviation, by hand



Score a model with K-fold and GroupKFold cross-validation, and report the mean with its standard deviation



Name three kinds of data leakage and use a checklist that blocks them

# Your three hours: ideas first, then the lab

| | | |
|---|---|---|
| 0:00 | 10 | Recall from Class 2, and why trust matters |
| 0:10 | 40 | Concepts, part 1: overfitting, validation curve, cross-validation |
| 0:50 | 25 | Concepts, part 2: data leakage, random seed, pipeline |
| 1:15 | 15 | Break |
| 1:30 | 70 | Lab: curve, cross-validation, leakage, seeds, records |
| 2:40 | 20 | The project, and how this class counts |

Concepts first, with no laptops. Then the lab, with the notebook open.

# Three questions from last time

1. Why is a majority-class baseline (a model that always predicts no) a poor model, even when its accuracy is about 80 percent?

2. In a confusion matrix, what is a false negative (FN) and what is a false positive (FP)?

3. What happened to recall and false positives when we lowered the decision threshold from 0.5 to 0.3?




Answer aloud first. Then we check together.

# Answers to the three questions from Class 2

| Question | Short answer |
| --- | --- |
| **Why is a majority-class baseline (a model that always predicts no) a poor model, even when its accuracy is about 80 percent?** | It finds none of the real positive cases, so its recall is 0. Accuracy hides that. |
| **In a confusion matrix, what is a false negative (FN) and what is a false positive (FP)?** | False negative: the model said no (negative) and the truth was yes (positive). False positive: the model said yes and the truth was no. |
| **What happened to recall and false positives when we lowered the decision threshold from 0.5 to 0.3?** | Recall stayed at 1.00. False positives went from 1 to 2. |

# Words we use today

| | Word | What it means |
|---|---|---|
|  | **Overfitting** | The model fits the noise in the training set (it nearly memorises it) and does worse on new rows. Underfitting is the opposite: too simple to learn the pattern. |
|  | **Cross-validation** | Scoring a model K times. The rows are cut into K parts called folds. Each fold takes one turn as the validation fold (held-out fold). |
|  | **Data leakage** | Information the model would not have at prediction time (held-out rows, or the target itself) reaches it while it is trained, so the score looks better than it really is. |
|  | **Bias-variance trade-off** | Too simple a model misses the pattern (high bias, underfitting). Too flexible a model chases noise (high variance, overfitting). |
|  | **Random seed** | A fixed number (random_state in scikit-learn) that makes a random choice repeat exactly. |

# A model that looks good on paper can fail on a new machine or site



A model is built from data on four crushers and then used on a fifth crusher.

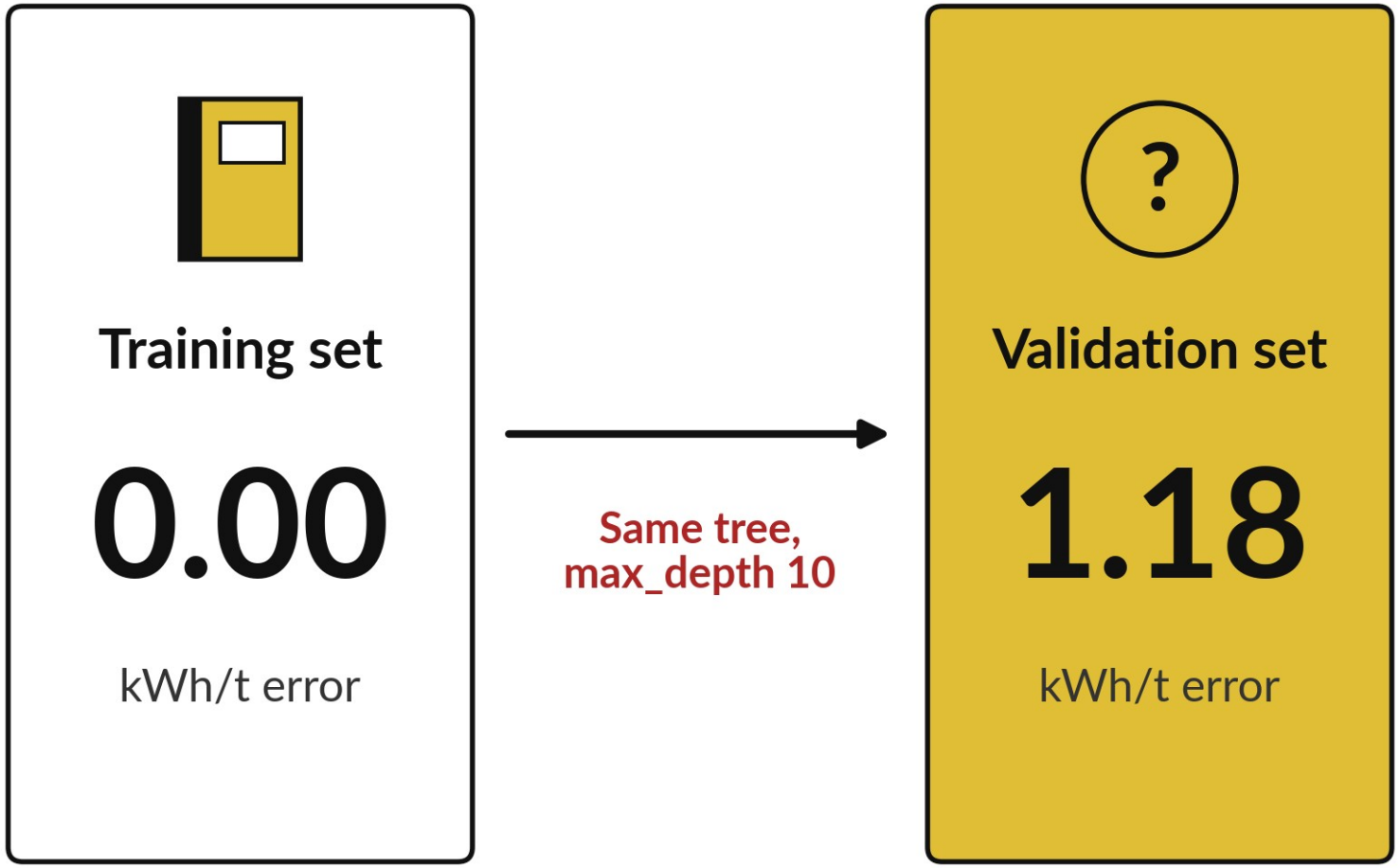


The power plan for the new pit is chosen from the model's prediction.



If the prediction is wrong, the energy bill holds a surprise.

# A model must do well on rows it has never seen

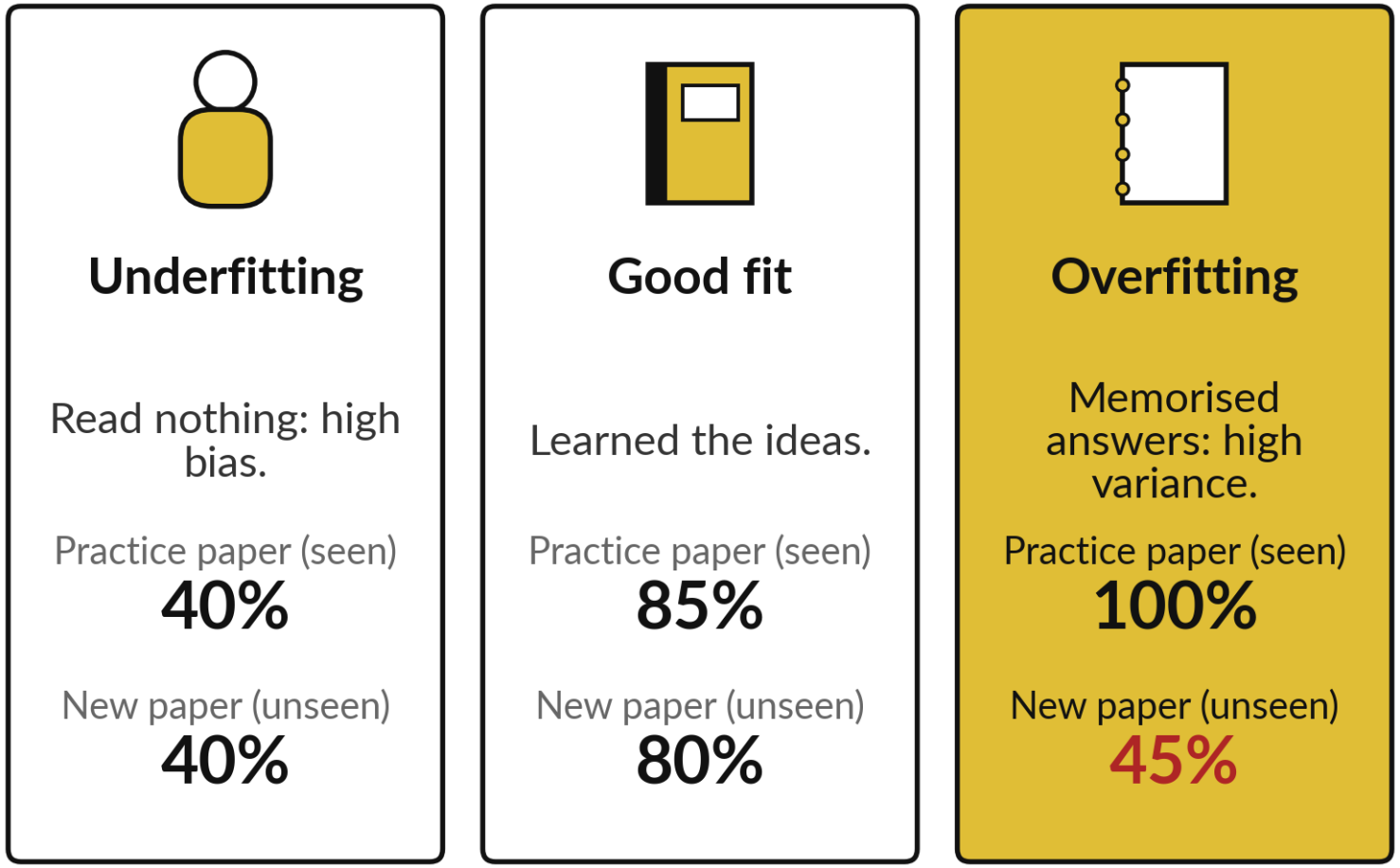



**Generalisation means doing well on held-out data: rows the model has never seen.**

The tree was trained on the training set, so its error there is almost zero.

The validation set was held out. On it the prediction error is far larger. The difference is the generalisation gap.

# Underfitting, a good fit and overfitting look like three students

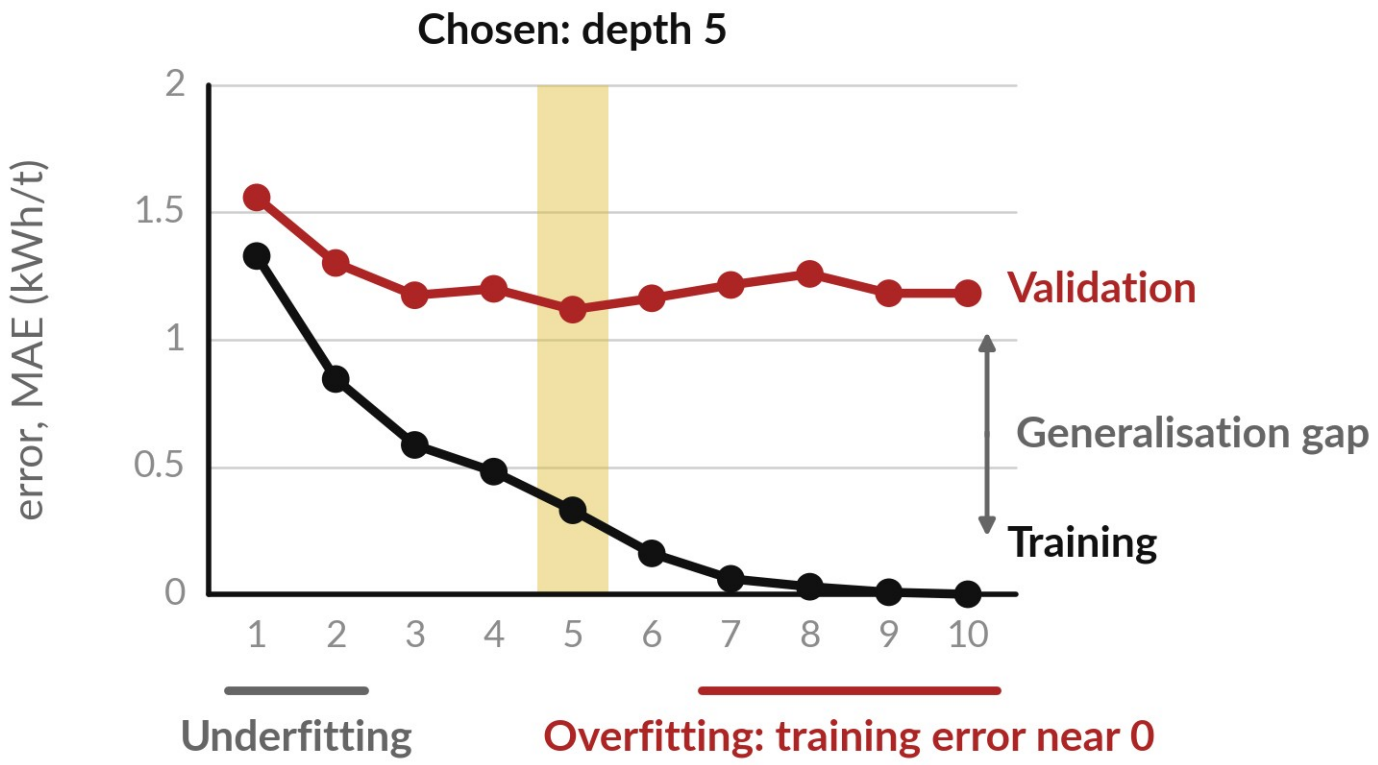



**The marks in the picture are illustrative (made up). Each student stands for one kind of model.**

Practice paper = training set. New paper = held-out rows.

Bias-variance trade-off: too simple a model misses the pattern, too flexible a model chases noise.

# The validation curve: as max_depth grows, training error keeps falling but validation error stops improving




**A validation curve plots both errors against max_depth, a hyperparameter (set before training).**

Each dot is one tree. Black: training error. Red: validation error. Between them: the gap.

Depths 1 and 2: both errors high. From depth 7: training error near 0. Gold: chosen depth 5.

# Training error, validation error and the generalisation gap are short formulas

Training error

$$\mathrm{MAE}_{\mathrm{train}} = \frac{1}{n_{tr}} \sum_{i \in \mathrm{train}} |y_i - \hat{y}_i|$$

Validation error

$$\mathrm{MAE}_{\mathrm{val}} = \frac{1}{n_{va}} \sum_{i \in \mathrm{val}} |y_i - \hat{y}_i|$$

| Symbol | What it means | In our data |
|---|---|---|
| **MAE** | Mean absolute error: the average size of the prediction error. The small label train or val says which rows | Depth 10: 0.00 and 1.18 kWh/t |
| **n** | How many rows. The small label tr means training and va means validation | 58 and 15 rows |
| **i** | A row number. The sum adds up the rows of that group | Row 1, row 2, and so on |
| **y, ŷ** | The real target of row i, and the model's prediction for it. The bars \| \| drop any minus sign | crusher energy demand and the tree's answer, in kWh/t |

# The generalisation gap is the validation error minus the training error

Generalisation gap

$$\text{gap} = \text{MAE}_{\text{val}} - \text{MAE}_{\text{train}}$$

Depth 10

$$\text{gap} = 1.18 - 0.00 = 1.18 \text{ kWh/t}$$

Depth 5

$$\text{gap} = 1.12 - 0.33 = 0.79 \text{ kWh/t}$$

| Symbol | What it means | In our data |
|---|---|---|
| **gap** | Generalisation gap: how much worse the model is on new rows | Depth 10: 1.18 kWh/t |
| **MAE_val** | Validation error, from the formula before | 1.18 kWh/t at depth 10 |
| **MAE_train** | Training error, from the formula before | 0.00 kWh/t at depth 10 |

Small gap: similar error on new rows. Big gap: overfitting. We choose max_depth by validation error.

# Five depths in one table: leaf nodes, errors and the generalisation gap

| max_depth | Leaf nodes | Training MAE | Validation MAE | Generalisation gap (validation minus training) |
|---|---|---|---|---|
| **1** | 2 | 1.33 | 1.56 | 0.23 |
| **2** | 4 | 0.85 | 1.30 | 0.46 |
| **4** | 14 | 0.48 | 1.20 | 0.72 |
| **5 (chosen)** | 25 | 0.33 | 1.12 | 0.79 |
| **10** | 58 | 0.00 | 1.18 | 1.18 |

MAE in kWh/t. A leaf node is a final node: it holds the prediction. The Class 1 linear regression model scores 0.67 kWh/t, better than every tree: the target is close to a straight line, and a tree only makes steps.

# We choose max_depth on the validation set and keep the test set sealed




The tree is trained here.

We compare depths here.

Sealed. Opened once, at the end.

**Each set is made of whole equipment groups. The picture shows the rows in each set.**

Only 15 validation rows, so a score can move by luck. Never tune on the test set.

Test set rule: open it once, after the model and any threshold are chosen. Refit the chosen model on E1 to E5, score it once on E6.

# Tuning a hyperparameter: choose the simplest max_depth within 5 percent of the lowest validation error

Tolerance limit

$$\text{limit} = 1.05 \times V_{\min}$$

Chosen depth

$$d^* = \text{smallest } d \text{ with } V(d) \leq \text{limit}$$

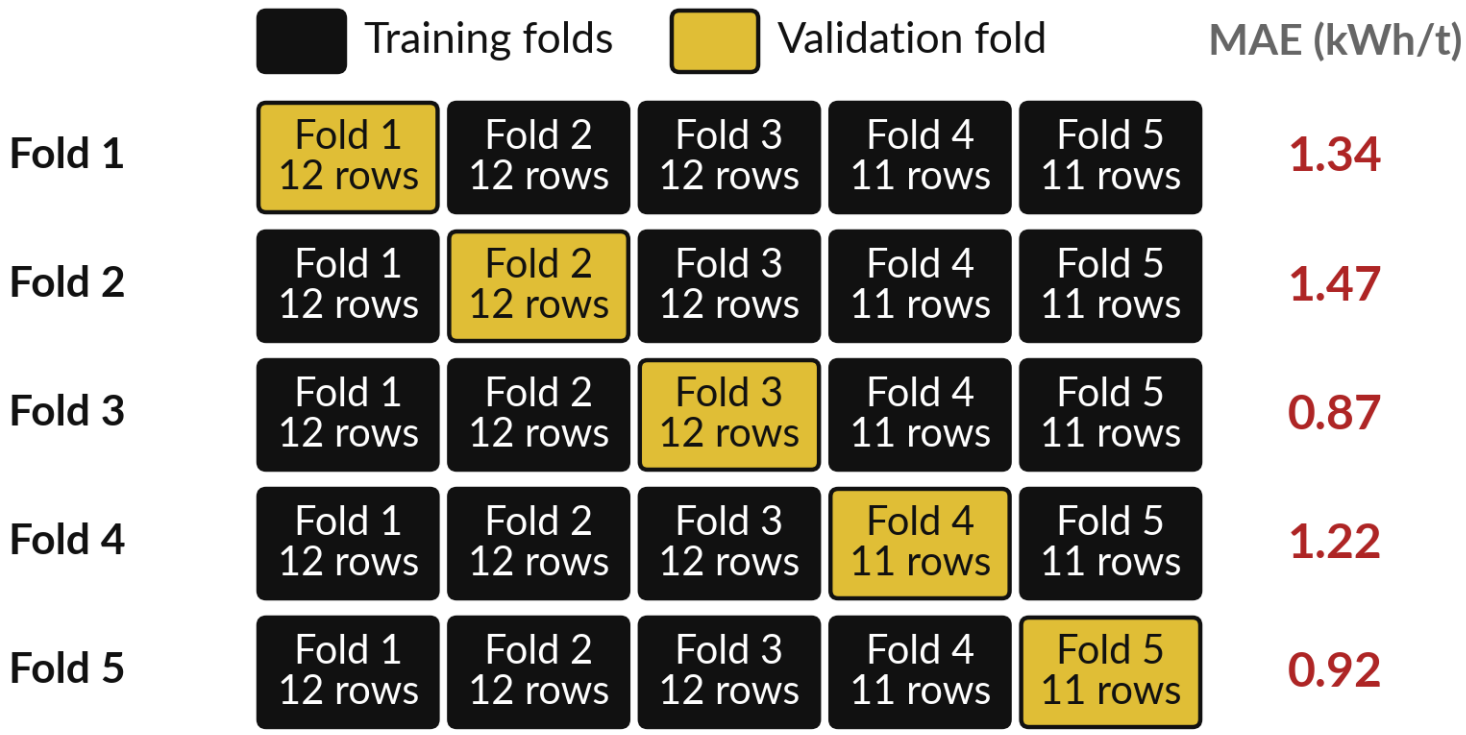
Our data

$$1.05 \times 1.118 = 1.174 \text{ kWh/t}, \; d^* = 5$$

| Symbol | What it means | In our data |
|---|---|---|
| **V(d)** | Validation MAE of the tree with max_depth d | 1.56 at depth 1, 1.18 at depth 10 |
| **V_min** | The lowest validation MAE over depths 1 to 10 | 1.118 kWh/t, at depth 5 |
| **limit** | The tolerance limit: the lowest value plus 5 percent | 1.174 kWh/t |
| **d*** | The chosen depth: the smallest d that passes | Depths that pass: 5, 6 |

Depth 3 (1.175) misses the limit (1.174) by a hair: the rule is sensitive. d* = chosen max_depth (parsimony).

# One split can be lucky, so K-fold cross-validation lets every fold take a turn as the validation fold




**Mean 1.16 kWh/t  Standard deviation 0.26 kWh/t**

**K-fold cross-validation cuts the 58 training rows into K = 5 folds (parts of the rows), in the order the rows appear (no shuffle).**

In each line, the tree is trained on 4 folds and scored on the gold validation fold.

Rows of E1 to E4 are interleaved: every fold holds all four groups (fold 1: E1 4, E2 4, E3 2, E4 2).

# K-fold gives k scores, and we report their mean and their standard deviation

Mean of the k scores

$$\bar{s} = \frac{1}{k} \sum_{j=1}^{k} s_j$$

Sample standard deviation of the k scores

$$\sigma_s = \sqrt{\frac{1}{k-1} \sum_{j=1}^{k} (s_j - \bar{s})^2}$$

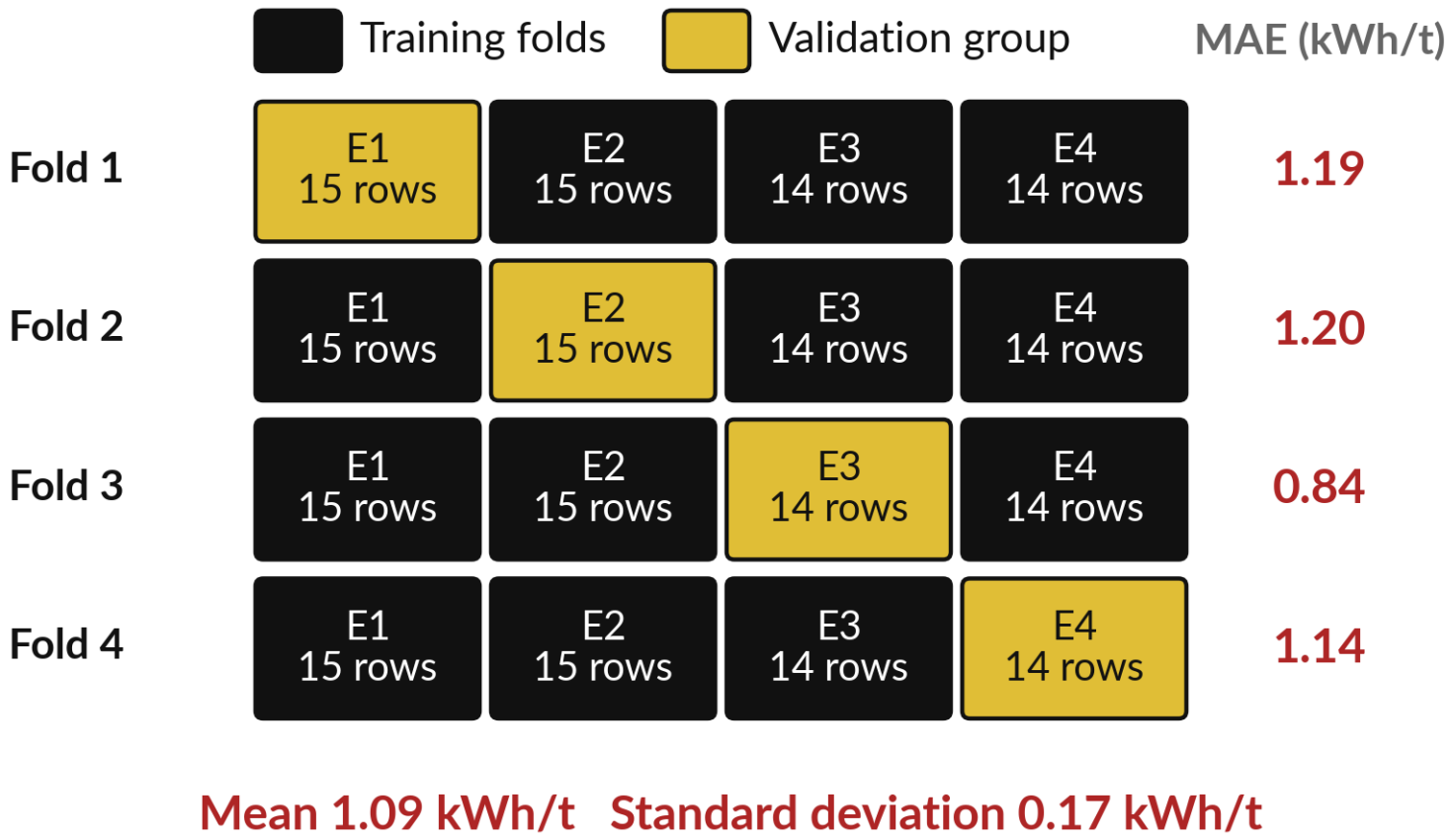
| Symbol | What it means | In our data |
|---|---|---|
| **s_j** | The MAE when fold j is the validation fold (j is the fold number) | 1.34, 1.47, 0.87, 1.22, 0.92 kWh/t |
| **k** | The number of folds, so the number of scores | 5 folds |
| **s̄** | The mean (average) of the k scores (s with a bar) | 1.16 kWh/t |
| **σ_s** | Sample standard deviation: typical distance of a score from the mean. np.std(scores, ddof=1) | 0.26 kWh/t |

# By hand: our five fold scores give their mean and standard deviation

| Fold | Score (MAE, kWh/t) | Distance from the mean | Squared distance |
|---|---|---|---|
| **1** | 1.34 | +0.17 | 0.030 |
| **2** | 1.47 | +0.31 | 0.096 |
| **3** | 0.87 | -0.29 | 0.087 |
| **4** | 1.22 | +0.06 | 0.003 |
| **5** | 0.92 | -0.24 | 0.060 |
| **Sum** | 5.82 | 0.00 | 0.276 |

Mean = 5.82 / 5 = 1.16 kWh/t. Sum of squares / (k - 1) = 0.276 / 4 = 0.069. Square root = 0.26 kWh/t, the standard deviation. Worked from unrounded scores, so the last digit can differ.

# GroupKFold holds out a whole equipment group in each fold, which is how the model is really used




**Mean 1.09 kWh/t  Standard deviation 0.17 kWh/t**

**GroupKFold keeps each group whole: each of E1 to E4 is held out once (scikit-learn picks the order).**

Plain K-fold put every crusher in every fold. Here the tree trains on three groups and is scored on the fourth, like a new site.

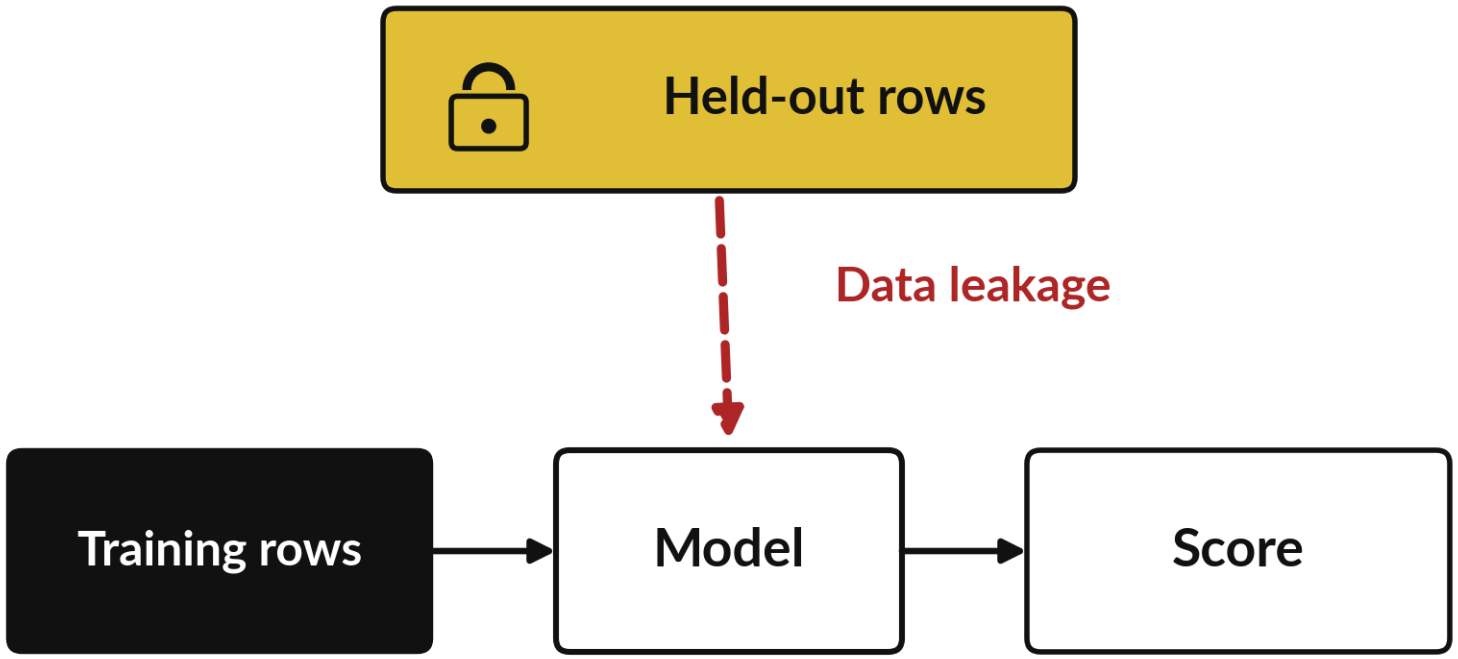
Score s_g: MAE on group g. Mean and sample standard deviation as before (divide by 3).

# The same max_depth 4 tree gets three different scores, depending on how we test it

| | Way of testing | Scores (MAE) | Mean | Standard deviation |
|---|---|---|---|---|
|  | **Single validation split (group E5)** | 1.20 | 1.20 | one score only |
|  | **K-fold, k = 5 (rows in order, no shuffle)** | 0.87 to 1.47 | 1.16 | 0.26 |
|  | **GroupKFold, k = 4** | 0.84 to 1.20 | 1.09 | 0.17 |

MAE in kWh/t. The single E5 score is within about one standard deviation of both means: the normal spread from one fold to another. The standard deviation shows how far to trust each mean.

# Data leakage is information the model would not have at prediction time reaching it during training




Normal path: training rows train the model, then it gets a score

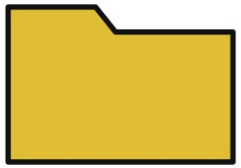
**Held-out rows must never reach the model**

**Data leakage: information the model would not have at prediction time (held-out rows, or the target itself) reaches it while it is trained.**

Held-out rows are the validation or test rows. The dashed red arrow is the leakage.

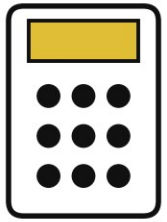
The score looks excellent, then the model fails on real new rows.

# Data leakage comes in three common kinds



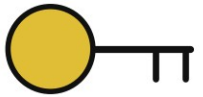
### 1 Train-test contamination

Validation rows are in the training set



### 2 Preprocessing leakage

Imputer (fills missing values) or scaler (rescales columns) fitted on all rows

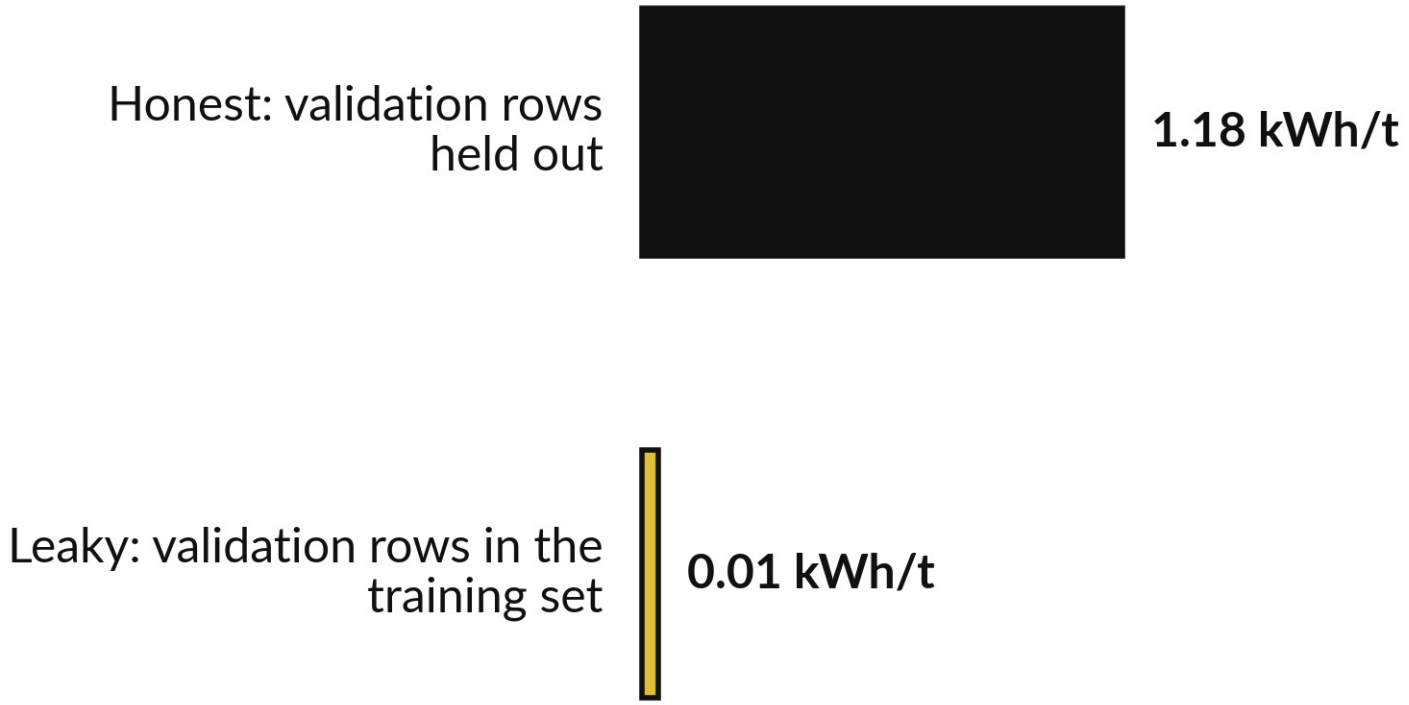


### 3 Target leakage

A feature is built from the target

All three make the score look too good.

# Train-test contamination: validation rows in the training set make the score look perfect

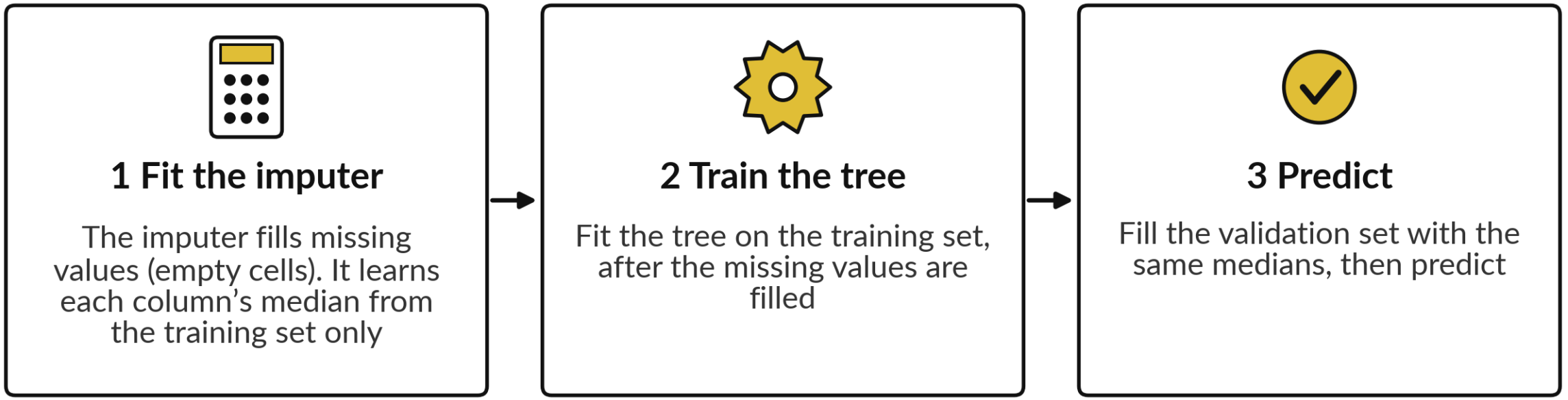



**Same max_depth 10 tree. Only the training set changed.**

In the leaky run the tree was already trained on the validation rows, so it reproduces their answers: it has already seen them.

The honest score is the real one.

# A Pipeline fits the imputer on the training set only, which prevents preprocessing leakage




Median of ambient_c: 26.07 from the training set, 25.37 from training plus validation rows (73 rows, leakage). Only one value is missing, so the score did not change.

# The pipeline rule: fit on the training set only, then apply the same numbers

Imputer: the median fill

$$m_j = \text{median}\{\, x_{ij} : i \in \text{train} \,\}$$

StandardScaler: mean, then z-score

$$\mu_j = \frac{1}{n_{tr}} \sum_{i \in \text{train}} x_{ij}, \qquad z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

| Symbol | What it means | In our data |
|---|---|---|
| **i, j, x_ij, n_tr** | Row i, column j, the value x_ij there, n_tr training rows. In preprocessing leakage, i also runs over validation rows | j = 1 load_pct, j = 2 ambient_c; n = 58 |
| **m_j** | Median of column j over the training rows | ambient_c: 26.07. Training plus validation: 25.37 (leakage) |
| **μ_j, σ_j** | Mean and standard deviation of column j, training rows (divide by n_tr, Class 2) | load_pct: 59.89 and 22.73 |
| **z_ij** | The z-score (standardised value) of row i in column j | Row 1: (59.15 - 59.89) / 22.73 = -0.03 |

# Target leakage: a feature built from the target gives a great score that cannot be repeated

| | A | B | C | D |
|---|---|---|---|---|
| | load_pct | ambient_c | target_rounded | target |
| 1 | 59.1 | 31.7 | 14 | 14.0 |
| 2 | 77.5 | 30.0 | 15 | 15.1 |
| 3 | 69.6 | 19.9 | 13 | 13.0 |

**With that feature a max_depth 4 tree scores 0.31 kWh/t. With honest features it scores 1.20 kWh/t.**

target_rounded is the target rounded to a whole number. At prediction time the target is unknown.

A feature recorded after the event is also target leakage.

# Keep this checklist: five questions before you trust a score



Was the score measured on held-out data (rows the model never saw)?



Were the imputer and the scaler fitted on the training set only (no preprocessing leakage)?



Is every feature available at the moment you need the prediction, and none built from the target (no target leakage)?



Is the test set (group E6) still sealed, to be opened once after the final choice?



Is the score a cross-validation mean, shown with its standard deviation?

# The same code can give different numbers, so we fix the random seed (random_state) and write it down

| random_state (the random seed) | max_depth 10 tree: validation MAE | Shuffled 5-fold: mean MAE | max_depth chosen by the 5 percent rule |
|---|---|---|---|
| **1** | 1.21 | 1.06 | 5 |
| **17** | 1.18 | 1.08 | 5 |
| **99** | 1.28 | 1.00 | 5 |

MAE in kWh/t. Only the seed changed. The chosen max_depth is the same for every seed here.

# Write down what you ran, so anyone can repeat it

**Run record**

Data file
*engineering_data.csv, 91 rows in the file*

Split
*Training E1 to E4, validation E5, test E6 sealed*

Random seed
*random_state = 17 (Step 5 tries 1, 99)*

Versions
*Python, NumPy, pandas, scikit-learn*

**Anyone with this record should get your numbers. After cleaning in Class 1, 88 rows remain: 58, 15, 15.**

The notebook prints the record for you.

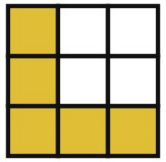
Paste it under every result you report.

# Trust comes from four habits



**Held-out data**

Score on rows the model never saw



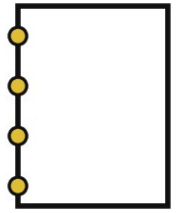
**Cross-validation**

Report mean and standard deviation



**No leakage**

Fit every preprocessing step on the training set



**Reproducibility**

Record seed, file, versions

The project needs all but cross-validation, which returns in Class 4.

# Read the workflow before you write it

```
pipeline = make_pipeline(imputer, tree_model)
group_kfold = GroupKFold(n_splits=4)
cv_scores = -cross_val_score(
    pipeline, X_train, y_train,
    groups=groups, cv=group_kfold,
    scoring="neg_mean_absolute_error")
print(cv_scores.mean(), cv_scores.std(ddof=1))
```

## In plain words

make_pipeline chains the imputer and the tree into one model (a Pipeline).

GroupKFold(n_splits=4): each fold holds out one equipment group.

cross_val_score runs the pipeline on every fold: one score per fold.

The scoring gives the MAE with a minus sign; the minus in front turns it back.

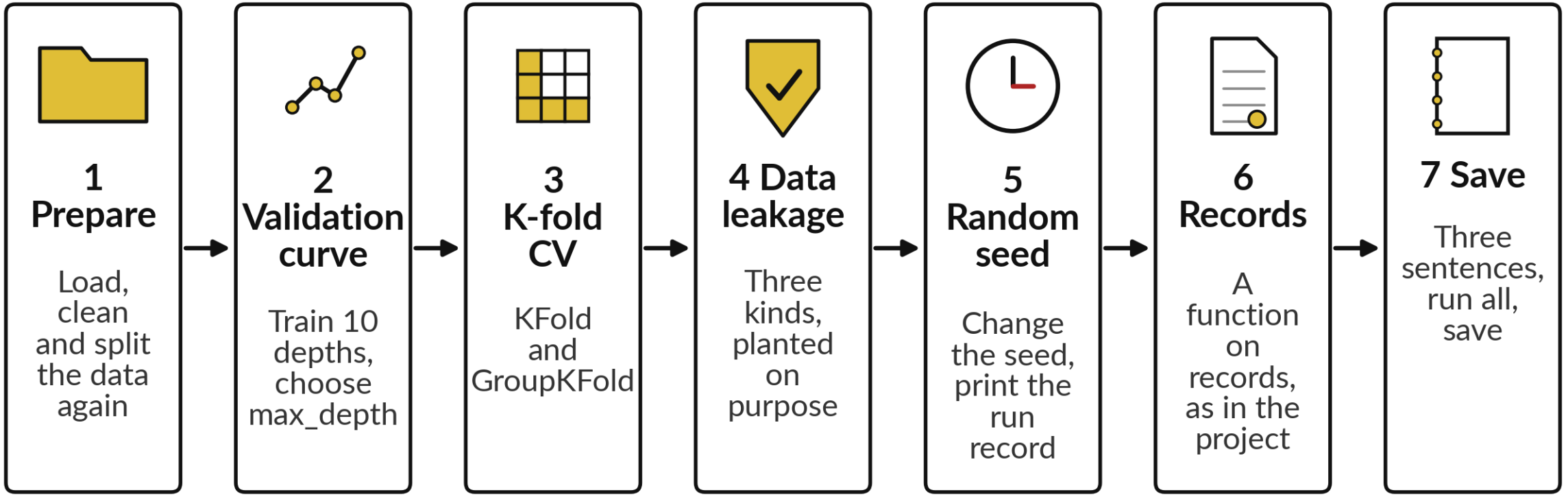
std(ddof=1): sample standard deviation (divide by k - 1).



# Part 2: the lab

Close the slides. Open the notebook. Same steps, your own data.

# The lab has seven steps, and the first six end with a check




In each step: predict, run, then read the output. Steps 1 to 6 end with a CHECK of your numbers.

# Check your numbers against these before you move on

| | What you print | You should see |
|---|---|---|
|  | **Rows: training, validation, test** | 58, 15, 15 |
|  | **Validation MAE at max_depth 1 and at 10** | 1.56 and 1.18 kWh/t |
|  | **Chosen max_depth** | 5 |
|  | **K-fold, k = 5: mean and standard deviation** | 1.16 and 0.26 |
|  | **GroupKFold, k = 4: mean and standard deviation** | 1.09 and 0.17 |
|  | **max_depth 10: honest and leaky validation MAE** | 1.18 and 0.01 |
|  | **max_depth 10 tree, random_state 1 and 99** | 1.21 and 1.28 |

MAE in kWh/t. Standard deviations divide by k - 1 (ddof=1). If your number differs, stop and ask. With another scikit-learn version a difference in the second decimal is fine.

# Four problems you will meet, what they mean and what to do

| What you see | What it means | What to do |
|---|---|---|
| **NameError: X_train** | Step 1 was not run | Run all cells from the top, in order |
| **An error that mentions groups** | GroupKFold needs the group labels (a groups argument) | Pass groups=groups |
| **All scores are negative** | scikit-learn gives errors a minus sign (neg_ scoring: bigger is better) | Put a minus in front: -scores |
| **Numbers differ a little from the slide** | The random seed (random_state) is missing or different | Use random_state=17 in every tree, except in Step 5 where the seeds 1 and 99 are tested on purpose |

# How this class counts towards your coursework



Nothing is submitted today. Block 2 (Classes 3 and 4) is practice with feedback. It does not count toward your final mark.



Its assignment (A9 from today, A10 and A11 from Class 4) closes on Friday 4 December, 08:00, Kampala time.

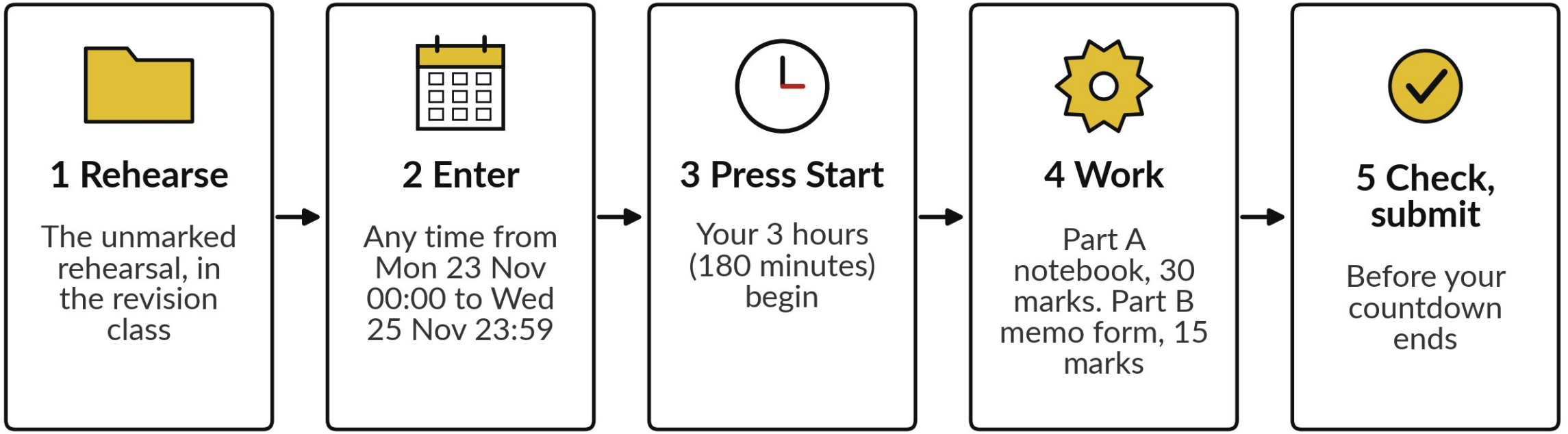


The Block 2 quiz opens in the last 20 minutes of Class 4, stays open 24 hours, and covers both classes.



Your final mark: A1 to A5, Q1 to Q5, Practical Test 1 and the Engineering ML project (40 percent).

# The project is one online sitting of 3 hours between 23 and 25 November




Individual, online from anywhere, marked by computer, 40 percent. Kampala time. Start by Wednesday 20:59 for the full 3 hours.

# You enter with your registration number, check your notebook, then submit from the last cell

| Step | What you do |
|---|---|
| **Enter** | Open your course entry link and type your registration number. Check that the name shown is yours, then press Start. A countdown shows your 3 hours. |
| **Check my notebook** | Restart session and run all, then run the Check my notebook cell. It runs your code again from the top and names any failing cell or missing function. It shows no mark. |
| **Submit** | Run the last cell, Submit your test, and press Submit test. Keep the receipt code. You may submit again before your deadline: the latest one on time is marked. |
| **Your result** | Your mark is shown after the window closes. |
| **Help** | Class channel, 08:00 to 20:00 each day. If something fails, post a screenshot at once: I can allow one fresh attempt. |

# The project tests data cleaning, three regression models, a warning question and one honest test

| Part | What it tests |
|---|---|
| **The data** | Real steel plant meter readings, one every 15 minutes for a year, the same file for everyone. You predict the energy used (kWh) in months the model has not seen. |
| **Part A, 30 marks** | Clean the data, two labelled charts, split by month (rule given), baseline, linear regression and decision tree in pipelines, MAE and RMSE, no leakage, the warning question, the test set opened once, results and functions. Not tested: cross-validation, Class 4. |
| **Warning question** | Logistic regression probabilities, then TP, FP, FN, TN, precision and recall at the fixed threshold 0.5. |
| **Part B, 15 marks** | The memo form: the chosen model, your numbers, limits, a next step. |
| **Rules** | Work alone, with no AI tool. Anyone may be asked online to explain their code. |

# Save the notebook, then do the rehearsal before the project opens

## What you keep from today

Your saved lab notebook, the five-question checklist, and the project window: online, Monday 23 to Wednesday 25 November.

### Our question today

*How do we know a model that predicts crusher energy demand will still work on a crusher it has never seen?*

### Next

The project, online, Mon 23 Nov 00:00 to Wed 25 Nov 23:59 (Kampala time). Class 4 is on Fri 27 Nov.

**Take away**

**Notebook**
*Restart, run all, save*

**max_depth**
*Chosen on the validation set*

**Rehearsal**
*Released today, not marked*

**Project**
*Online, 23 to 25 November*